

Day 31



The Promise API:

There are two types of methods:

A) Instance Methods

(Methods used *on a promise you created*)

Instance Methods

- **then()** → success
- **catch()** → error
- **finally()** → always

B) Static Methods

(Methods used *on the Promise class itself*, like `Promise.all()`)

Static Methods

- **Promise.resolve()** → make resolved promise
- **Promise.reject()** → make rejected promise
- **Promise.all()** → wait for all (fails if one fails)
- **Promise.allSettled()** → wait for all (never fails)
- **Promise.race()** → first finished
- **Promise.any()** → first successful

Example: basic example to print the resolved value of the promises in the console.

```
JS main.js > p2.then() callback
1  let p1 = new Promise((resolve, reject) => {
2    |   setTimeout(() => {
3    |     |   resolve("Value 1");
4    |     |   }, 1000);
5    |   });
6
7  let p2 = new Promise((resolve, reject) => {
8    |   setTimeout(() => {
9    |     |   resolve("Value 2");
10   |     |   }, 2000);
11   |   });
12
13 let p3 = new Promise((resolve, reject) => {
14   |   setTimeout(() => {
15   |     |   resolve("Value 3");
16   |     |   }, 3000);
17   |   });
18
19 p1.then((value) => {
20   |   console.log(value);
21   |   });
22 p2.then((value) => {
23   |   console.log(value);
24   |   });
25 p3.then((value) => {
26   |   console.log(value);
27   |   });
28
29
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS E:\JavaScript\Day31-40\Day31> node main.js
Value 1
Value 2
Value 3
```

Here, after 1s, Value 1 is printed, after 2s Value 2 is printed and after 3s, Value 3 is printed.

Example: doing the same thing we did above using the `promise.all()`

```
30 let p1 = new Promise((resolve, reject) => {
31   setTimeout(() => {
32     resolve("Value 1");
33   }, 1000);
34 });
35
36 let p2 = new Promise((resolve, reject) => {
37   setTimeout(() => {
38     resolve("Value 2");
39   }, 2000);
40 });
41
42 let p3 = new Promise((resolve, reject) => {
43   setTimeout(() => {
44     resolve("Value 3");
45   }, 3000);
46 });
47
48 let promise_all = Promise.all([p1,p2,p3]);
49 promise_all.then((values) => {
50   console.log(values);
51 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[ 'Value 1', 'Value 2', 'Value 3' ]
```

But what if one of the promise get rejected? Will `promise.all()` still works? No!

```
54 let p1 = new Promise((resolve, reject) => {
55   setTimeout(() => {
56     reject("Some error occurred in p1");
57   }, 1000);
58 });
59
60 let p2 = new Promise((resolve, reject) => {
61   setTimeout(() => {
62     resolve("Value 2");
63   }, 2000);
64 });
65
66 let p3 = new Promise((resolve, reject) => {
67   setTimeout(() => {
68     resolve("Value 3");
69   }, 3000);
70 });
71
72 let promise_all = Promise.all([p1,p2,p3]);
73 promise_all.then((values) => {
74   console.log(values);
75 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day31-40\Day31> node main.js
new UnhandledPromiseRejection(reason);
^
UnhandledPromiseRejection: This error originated either by throwing inside of an async function without a catch block, or by rejecting a promise
(). The promise rejected with the reason "Some error occurred in p1".
```

How to deal with this? We can use `.catch()` or can use `Promise.allSettled()` as shown below:

```
77 let p1 = new Promise((resolve, reject) => {
78     setTimeout(() => {
79         reject("Some error occurred in p1");
80     }, 1000);
81 });
82
83 let p2 = new Promise((resolve, reject) => {
84     setTimeout(() => {
85         resolve("Value 2");
86     }, 2000);
87 });
88
89 let p3 = new Promise((resolve, reject) => {
90     setTimeout(() => {
91         resolve("Value 3");
92     }, 3000);
93 });
94
95 let promise_all = Promise.allSettled([p1,p2,p3]);
96 promise_all.then((values) => {
97     console.log(values);
98 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day31-40\Day31> node main.js
[
  { status: 'rejected', reason: 'Some error occurred in p1' },
  { status: 'fulfilled', value: 'Value 2' },
  { status: 'fulfilled', value: 'Value 3' }
]
```

Now, what if we wish that the promise which first get resolved will get printed? Then we can use `Promise.race()`:

```
100 let p1 = new Promise((resolve, reject) => {
101     setTimeout(() => {
102         resolve("Value 1");
103     }, 111000);
104 });
105
106 let p2 = new Promise((resolve, reject) => {
107     setTimeout(() => {
108         resolve("Value 2");
109     }, 2000);
110 });
111
112 let p3 = new Promise((resolve, reject) => {
113     setTimeout(() => {
114         resolve("Value 3");
115     }, 3000);
116 });
117
118 let promise_all = Promise.race([p1,p2,p3]);
119 promise_all.then((values) => {
120     console.log(values);
121 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day31-40\Day31> node main.js
Value 2
```

What if the fastest one giving an error? Then `.race()` will show error. How to deal then? We can use `.any()`:

```
100 let p1 = new Promise((resolve, reject) => {
101   setTimeout(() => {
102     resolve("Value 1");
103   }, 10000);
104 });
105
106 let p2 = new Promise((resolve, reject) => {
107   setTimeout(() => {
108     reject("Some error occurred in p2");
109   }, 2000);
110 });
111
112 let p3 = new Promise((resolve, reject) => {
113   setTimeout(() => {
114     resolve("Value 3");
115   }, 3000);
116 });
117
118 let promise_all = Promise.any([p1,p2,p3]);
119 promise_all.then((values) => {
120   console.log(values);
121 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day31-40\Day31> node main.js
Value 3

--The End--